EE2213: Introduction to Artificial Intelligence

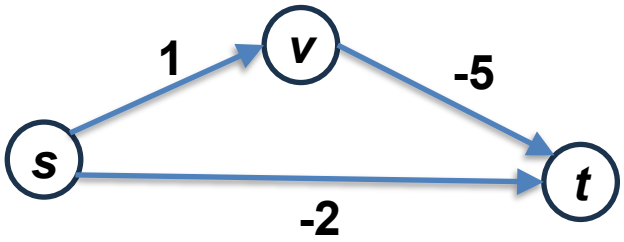
Tutorial 2 (Lectures 2 & 3)

Dr. Shaojing Fan
fanshaojing@nus.edu.sg

Review: Quick question 2

What is the computed shortest distance from *s* to *t*, when running Dijkstra's algorithm on this graph? Assuming it treats the graph normally despite negative edges.

(a) 1
(b) -4
(c) -2 ✓
(d) ∞



Refer to Lecture3_Dijkstra_negative_edges_Example.pdf

Dijkstra's algorithm (uniform-cost search): Pseudocode

- Start with a **frontier (priority queue)** containing the initial state (source), with path cost = 0.
- Start with an **empty explored set**.
- Repeat:
  - -- If the frontier is empty, then no solution.
  - -- Remove the frontier node with the lowest path cost from the source.
  - -- If this node is the goal, return the solution.
  - -- Add the node to the explored set.
  - -- Expand the node:
    - For each child, compute its total path cost from the source.
    - If the child is not in the frontier or explored set, **add it to the frontier** with its cost.
    - If the child is already in the frontier with a **higher cost**, update it with the **lower cost**.

A node is *marked as visited* once it is removed from the frontier (if not goal)

Dijkstra's algorithm has a greedy nature

Greedy algorithms:

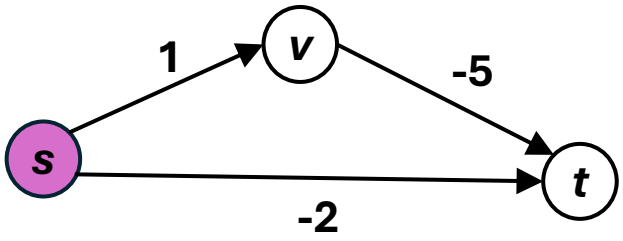
Make *locally* optimal choices at each step to achieve a *globally* optimal solution.

Dijkstra's algorithm is **greedy** because at each step, it selects the node with the **smallest known distance** from the source — assuming that this choice will lead to the overall shortest path.

It **does not backtrack** or reconsider previous choices, which is a hallmark of greedy algorithms.

However, it's also **guaranteed to find the shortest path** in graphs with **non-negative edge** weights, so the greedy choice works correctly in that setting.

Main loop: Iteration 1



frontier
(priority queue)

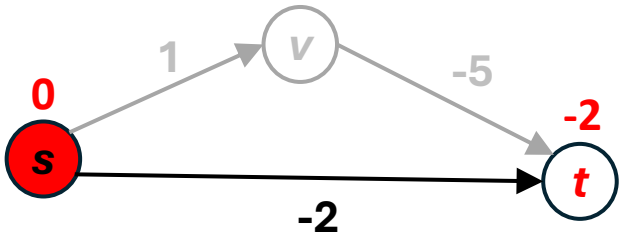
node	dist
s	0

node	shortest distance from *a*	previous node
s	0	--
v	∞	--
t	∞	--

Processed Nodes = { }

Unprocessed Nodes = {s, v, t}

Main loop: Iteration 1



$dist[t] = -2$

frontier
(priority queue)

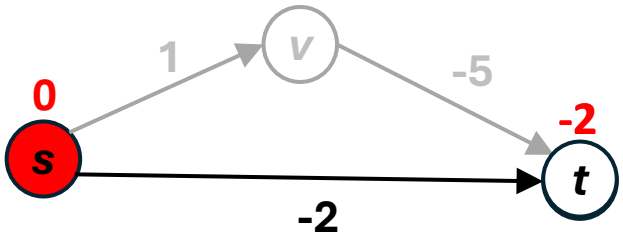
node	dist

node	shortest distance from *a*	previous node
s	0	--
v	∞	--
t	−2	*s*

Processed Nodes = { }

Unprocessed Nodes = {s, v, t}

Main loop: Iteration 1



$$dist[t] = -2$$

frontier
(priority queue)

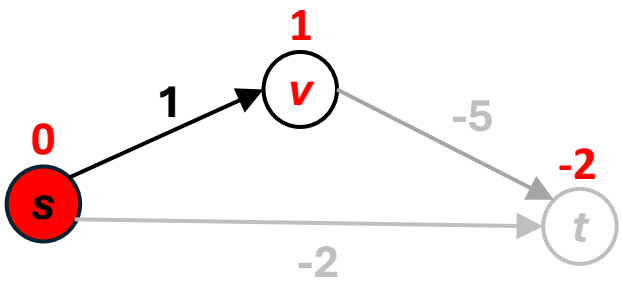
node	dist

node	shortest distance from *a*	previous node
s	0	--
v	∞	--
t	−2	*s*

Processed Nodes = { }

Unprocessed Nodes = {s, v, t}

Main loop: Iteration 1



$dist[v] = 1$

frontier (priority queue)

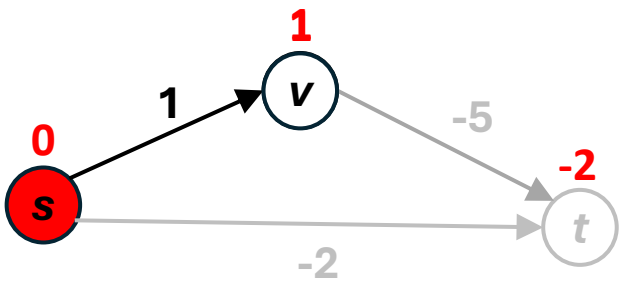
node	dist
t	-2

Processed Nodes = { }

node	shortest distance from *a*	previous node
s	0	--
v	1	*s*
t	-2	*s*

Unprocessed Nodes = {s, v, t}

Main loop: Iteration 1



$dist[v] = 1$

frontier (priority queue)

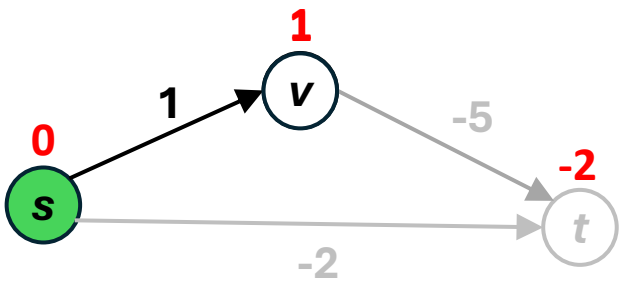
node	dist
t	-2

Processed Nodes = { }

node	shortest distance from *a*	previous node
s	0	--
v	1	*s*
t	-2	*s*

Unprocessed Nodes = {s, v, t}

Main loop: Iteration 1



frontier
(priority queue)

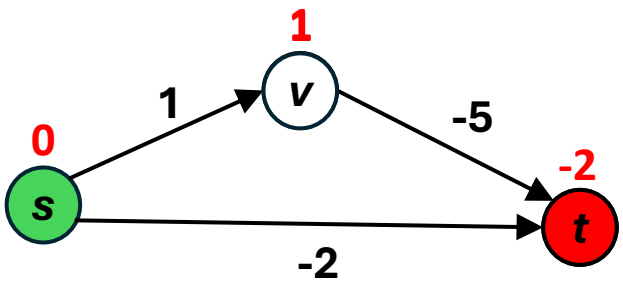
node	dist
t	-2
v	1

node	shortest distance from *a*	previous node
s	0	--
v	1	*s*
t	-2	*s*

Processed Nodes = {s}

Unprocessed Nodes = {v, t}

Main loop: Iteration 2



frontier
(priority queue)

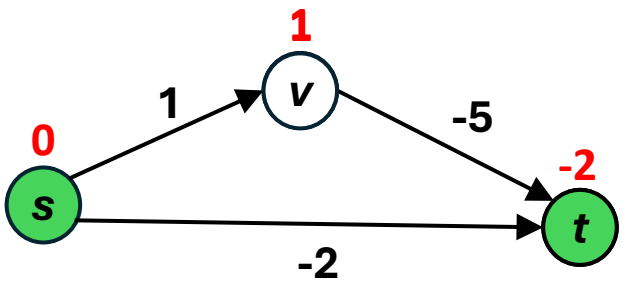
node	dist
t	-2
v	1

node	shortest distance from *a*	previous node
s	0	--
v	1	*s*
t	−2	*s*

Processed Nodes = {s}

Unprocessed Nodes = {v, t}

Main loop: Iteration 2



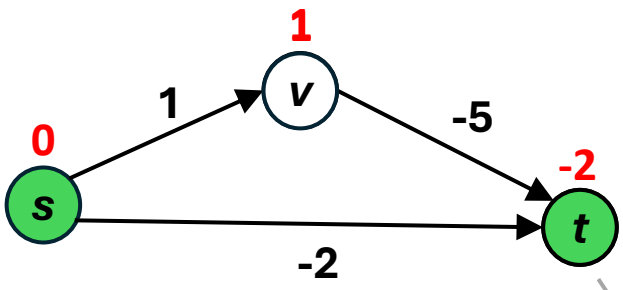
frontier
(priority queue)

node	dist
v	1

node	shortest distance from *a*	previous node
s	0	--
v	1	*s*
t	−2	*s*

Processed Nodes = {s, t } **Unprocessed Nodes = {v}**

Main loop: Iteration 2



frontier
(priority queue)

node	dist
v	1

node	shortest distance from *a*	previous node
s	0	--
v	1	*s*
t	−2	*s*

Processed Nodes = {s, t }

Unprocessed Nodes = {v}

Finalized!

Dijkstra's algorithm (uniform-cost search): Pseudocode

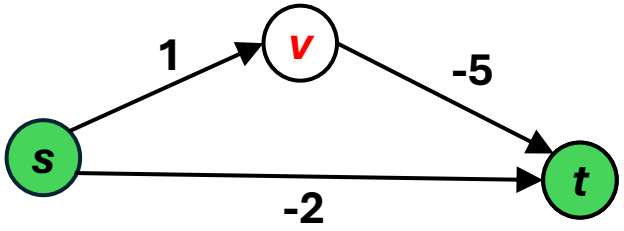
- Start with a **frontier (priority queue)** containing the initial state (source), with path cost = 0.
- Start with an **empty explored set**.
- Repeat:
  - -- If the frontier is empty, then no solution.
  - -- Remove the frontier node with the lowest path cost from the source.
  - -- If this node is the goal, return the solution.
  - -- Add the node to the explored set.
  - -- Expand the node:
    - For each child, compute its total path cost from the source.
    - If the child is not in the frontier or explored set, **add it to the frontier** with its cost.
    - If the child is already in the frontier with a **higher cost**, update it with the **lower cost**.

A node is *marked as visited* once it is removed from the frontier (if not goal)

Why Dijkstra's algorithm fails in negative edges

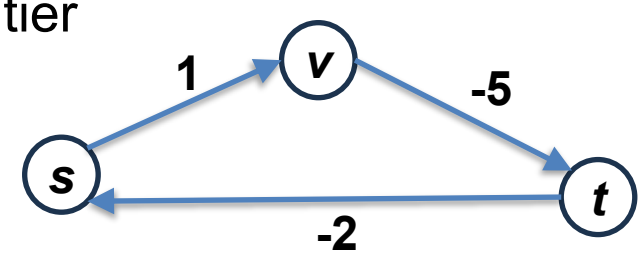
Assumes final path upon node removal:
Dijkstra assumes that once a node (like *t*) is removed from the frontier, the shortest path from the start node to it has been *finalized* (i.e., the node will not be added to the frontier again). This assumption breaks down when there are negative edge weights.

Fails to update better paths later:
Since *t* is first reached via $s \rightarrow t$ with cost –2, Dijkstra never enqueues t again, that is, it never reconsiders the better path $s \rightarrow v \rightarrow t$, which has a total cost of –4.



Can we modify it to handle negative edges?

If we revise Dijkstra's algorithm to allow revisiting nodes (i.e., allow nodes to be added to the frontier multiple times), it may lead to infinite loops in graphs with negative cycles.



Dijkstra's algorithm relies on non-negative edges.

For graph with negative edges or cycles, the Bellman-Ford algorithm is specifically designed to handle them properly (not covered in this course)

Further reading (beyond syllabus): https://en.wikipedia.org/wiki/Bellman%E2%80%93Ford_algorithm

Q1. Which of the following statements about tree search and graph search are correct? Select all that apply.

A. If a state is reachable via multiple paths, tree search will create two separate nodes corresponding to each path

B. Tree search checks if a node has been visited before adding it to the search queue.

C. Tree search is more efficient than graph search.

D. Graph search checks if a node has been visited before adding it to the search queue.

Recap: Graph search vs Tree search

NUS National University of Singapore

Tree search

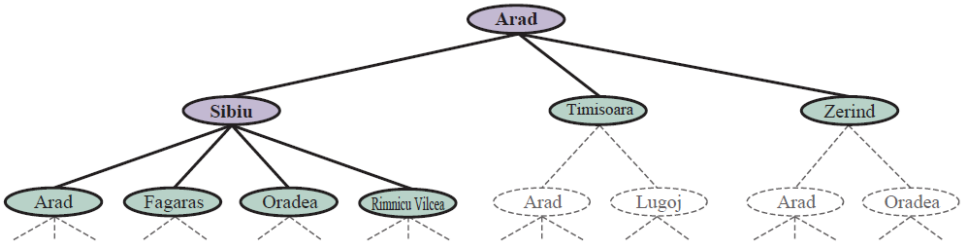
```
function TREE-SEARCH(problem, strategy) return a solution, or failure
        Initialize frontier to the initial state of the problem
        loop do
                if the frontier is empty then return failure
                choose a node from the frontier for expansion according to strategy,
                and remove it from frontier
         if the node contains goal state then return the corresponding solution
                else expand the node, and add the resulting child nodes to the frontier
         end
```

Graph search

```
function GRAPH-SEARCH(problem, strategy) return a solution, or failure
        Initialize frontier to the initial state of the problem
        Initialize the explored set to be empty
        loop do
                if the frontier is empty then return failure
                choose a node from the frontier for expansion according to strategy,
                and remove it from frontier
         if the node contains goal state then return the corresponding solution
         add the node to the explored set
         expand the node, and add the resulting nodes to the frontier
            only if not in the frontier or explored set
         end
```

Reference: https://www.youtube.com/watch?v=Y1VywhuRFIs

Q1. Which of the following statements about tree search and graph search are correct? Select all that apply.



A. If a state is reachable via multiple paths, tree search will create two separate nodes corresponding to each path ✓

B. Tree search checks if a node has been visited before adding it to the search queue. Graph search!

C. Tree search is more efficient than graph search. Tree search is less efficient as it allows revisiting the same nodes.

D. Graph search checks if a node has been visited before adding it to the search queue. ✓

Q2. Which of the following is true about greedy best-first search? Select all that apply.

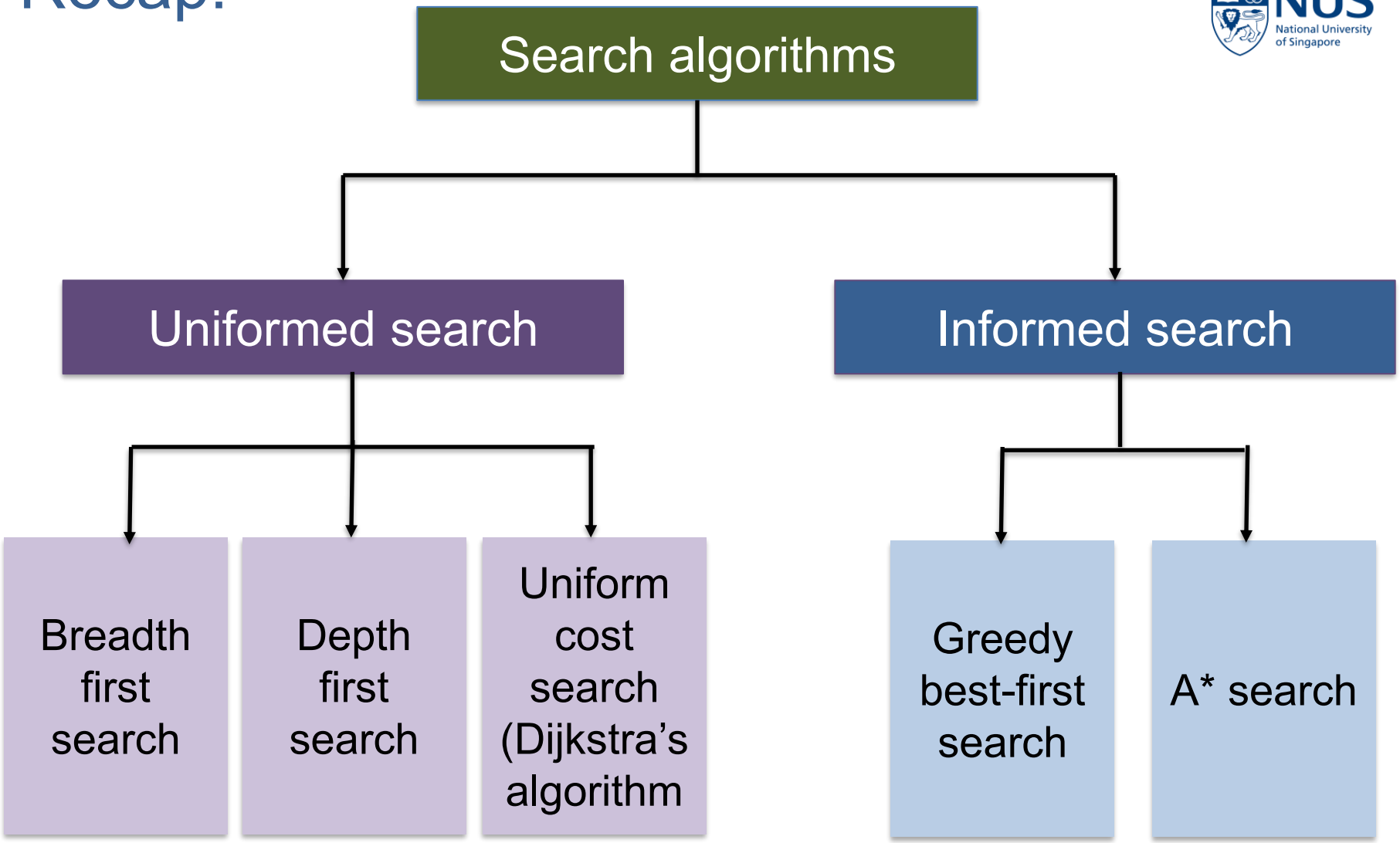
A. It guarantees both completeness and optimality when using a well-designed heuristic.

B. It expands nodes based solely on the lowest cumulative path cost from the start state.

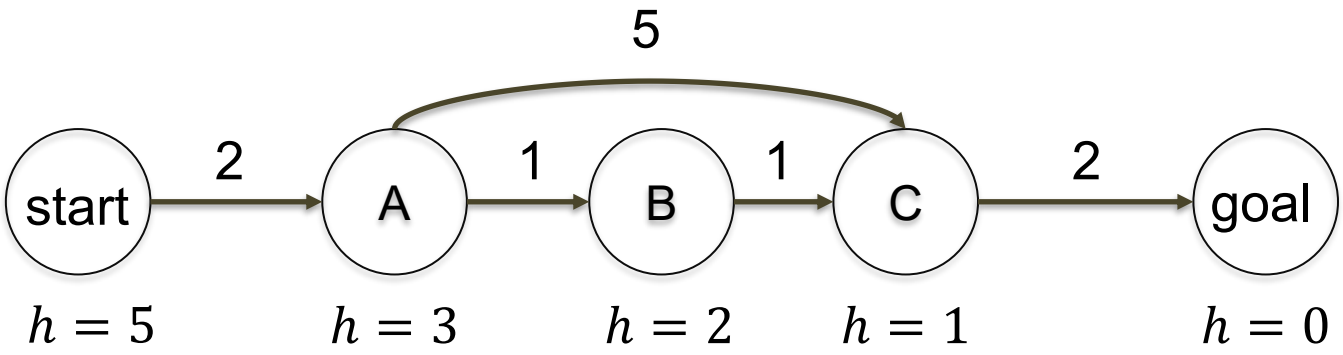
C. It expands the node that appears closest to the goal, based only on the heuristic function.

D. It behaves identically to uniform-cost search when edge costs are equal.

Recap:



Greedy best-first search is not optimal



Greedy best-first search: start → A → C → goal, path cost = 9

Actual optimal path: start → A → B → C → goal, path cost = 6

Greedy best-first search uses *only the heuristic* to choose the node that seems closest to the goal, *ignoring the actual cost* from the start—so it can miss the cheapest path.

Greedy best-first search isn't optimal even with a well-defined heuristic

In this maze, each grid is labeled with its Manhattan distance to the goal as a heuristic. Use greedy best-first search to find the shortest path from A to B.

	10	9	8	7	6	5	4	3	2	1	B
	11										1
	12		10	9	8	7	6	5	4		2
	13		11						5		3
	14	13	12		10	9	8	7	6		4
			13		11						5
A	16	15	14		12	11	10	9	8	7	6

Greedy best-first search isn't optimal even with a well-defined heuristic

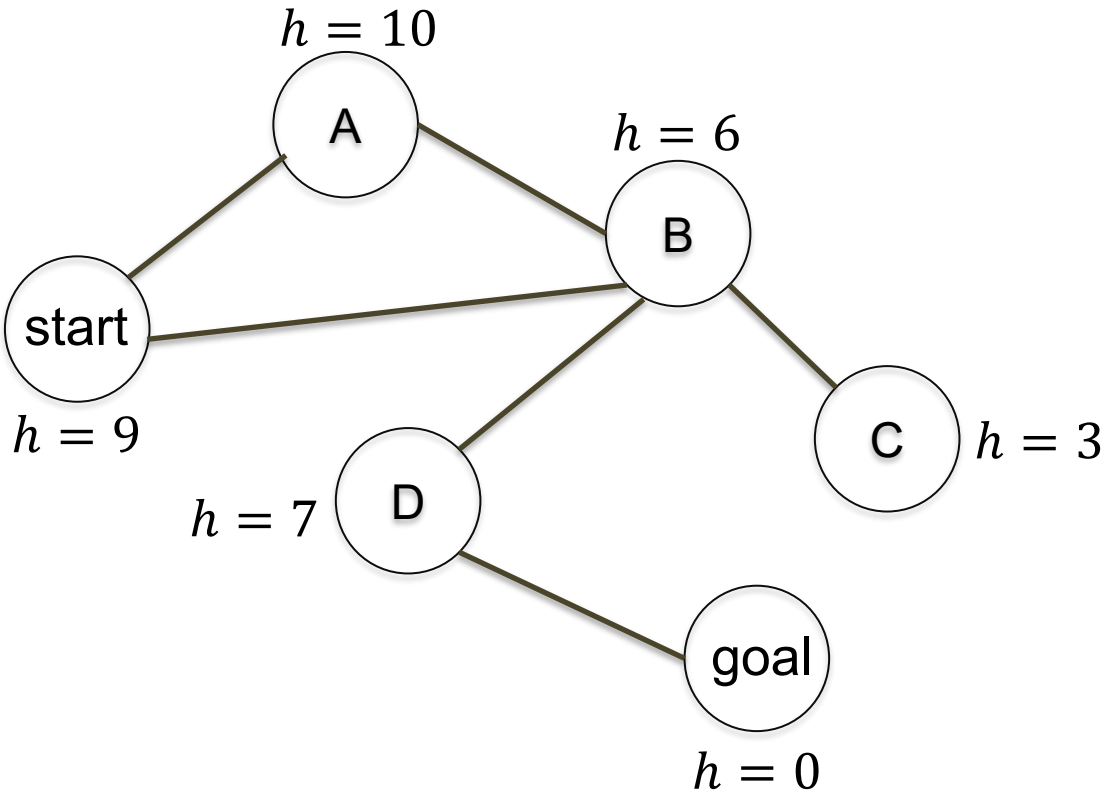
In this maze, each grid is labeled with its Manhattan distance to the goal as a heuristic. Use greedy best-first search to find the shortest path from A to B.

	10	9	8	7	6	5	4	3	2	1	B
	11										1
	12		10	9	8	7	6	5	4		2
	13		11						5		3
	14	13	12		10	9	8	7	6		4
			13		11						5
A	16	15	14		12	11	10	9	8	7	6

The problem isn't just the heuristic — it's the nature of greedy best-first search itself!

Is greedy best-first search complete?

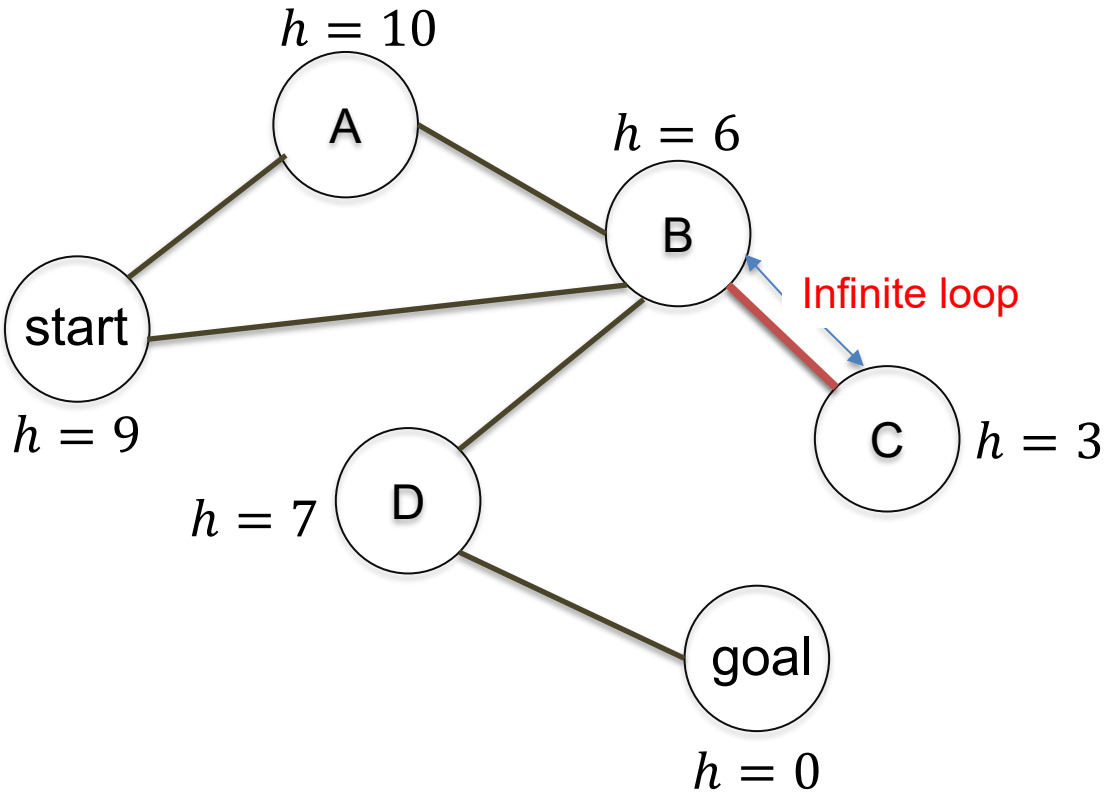
Use greedy best-first search to find a path from the start to the goal, assuming a tree search strategy.



Each node is labeled with its heuristic estimate $h(n)$ (shown beside the node),

Is greedy best-first search complete?

Use greedy best-first search to find a path from the start to the goal, assuming a tree search strategy.



Each node is labeled with its heuristic estimate $h(n)$ (shown beside the node),

Properties of greedy best-first search

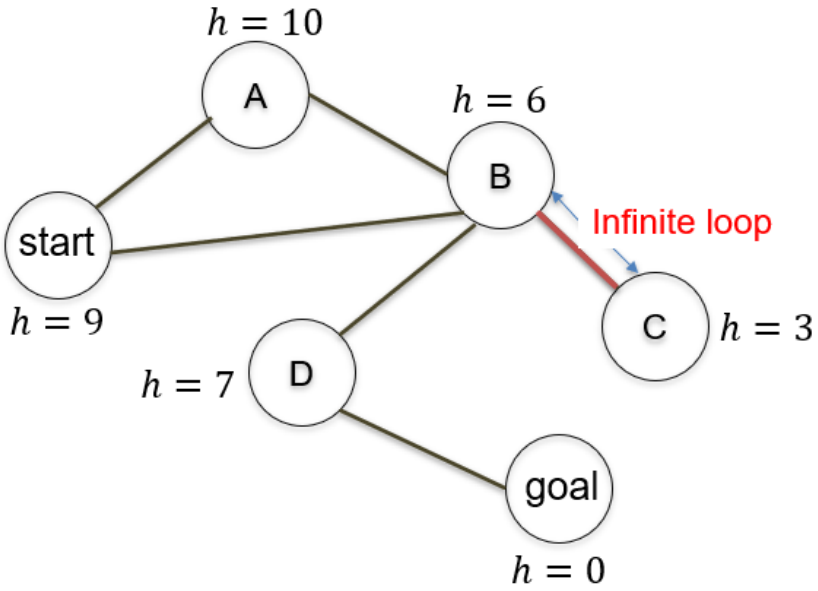
Complete? No–can get stuck in loops

Optimal? No–not guaranteed to render lowest cost solution.

Completeness: Does the algorithm always find a solution if one exists?
Optimality: Does it find the lowest-cost (best) solution among all possible ones?

Limitation of greedy best-first search

It *only considers* the estimated cost to the goal (heuristic) and *ignores* the actual cost already incurred to reach the current node. This can cause it to get trapped in local minima or follow paths that seem promising at first but stray from the optimal solution.



	10	9	8	7	6	5	4	3	2	1	B
	11										1
	12		10	9	8	7	6	5	4		2
	13		11						5		3
	14	13	12		10	9	8	7	6		4
			13		11						5
A	16	15	14		12	11	10	9	8	7	6

Q2. Which of the following is true about greedy best-first search? Select all that apply.

A. It guarantees both completeness and optimality when using a well-designed heuristic.

B. It expands nodes based solely on the lowest cumulative path cost from the start state.

C. It expands the node that appears closest to the goal, based only on the heuristic function. ✓

D. It behaves identically to uniform-cost search when edge costs are equal.

Q2 Takeaway

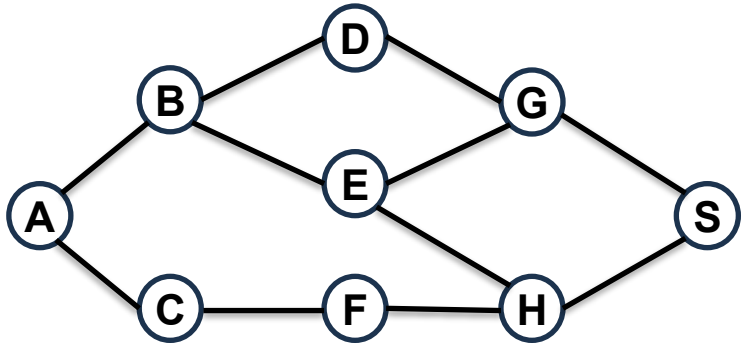
Greedy best-first search is:

- **Heuristic-guided**: At each step, it expands the node that **appears closest to the goal** based on the heuristic estimate (i.e., the smallest $h(n)$ value).
- **Not optimal**: It ignores the actual cost of the path taken so far ($g(n)$), so it may return a **suboptimal** solution.
- **Not always complete**: If the search space is infinite, contains cycles, or the heuristic is poor, it may get **stuck in a loop** or fail to find a solution, even if one exists.
- *Fast in practice* (sometimes): With a good heuristic, it can reach the goal quickly, but there are *no guarantees* of finding the best or even any solution.

Q3. You are given the following undirected graph.

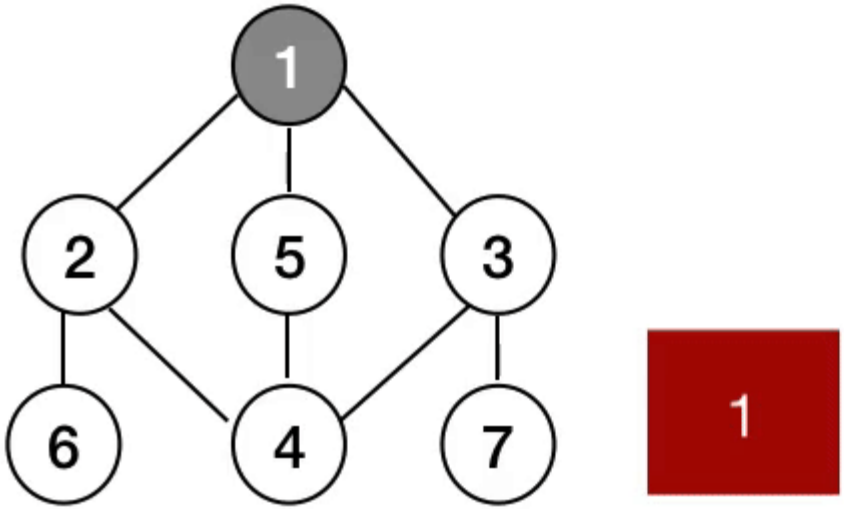
(i) Write a Python program to perform a Breadth-First Search (BFS) traversal starting from node ‘A’. Print the order in which nodes are visited.

(ii) Write a Python program to perform a Depth-First Search (DFS) traversal starting from node ‘A’. Print the order in which nodes are visited.



Note: The order of nodes visited may differ depending on the order neighbors are processed.

Recap: BFS



Credit: https://medium.com/@dillihangrae/searching-in-ai-dfs-bfs-7417d34e8113

Q3. (i) BFS

```python
def bfs(graph, start):
    visited = []    # List to keep track of visited nodes
    queue = []      # Queue to hold nodes to explore
    sequence = []  # List to store the visiting sequence

    visited.append(start)
    queue.append(start)

    while queue:
        node = queue.pop(0)
        print(node)
        sequence.append(node)

        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)
    return sequence
```

Create an empty list to keep track of explored (visited) nodes
Create an empty list serve as the frontier, which acts as a **queue**
Create an empty list to store the visited nodes *in the order they are explored*

Add the starting node to the list of visited nodes.
Add the starting node to the frontier (queue) to begin the search

While frontier is not empty, repeat:

-- Pop the earliest added node (i.e., the first node in the queue) from the frontier (**FIFO order**).

-- Add it to the sequence, and start to expand the node

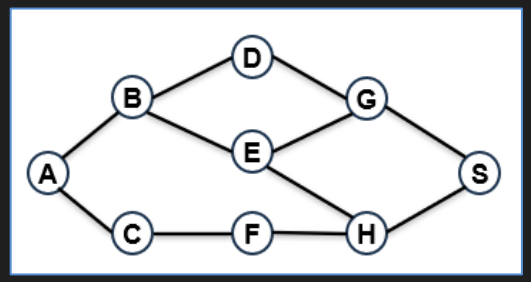
-- For each neighbor of the node:

- If the neighbor has not been visited,
- Mark it as visited and add it to the end of the queue.

Q3. (i) BFS (cont.)

```python
# Define the graph using a dictionary.
# Each node maps to a list of its connected neighbors.
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B', 'G'],
    'E': ['B', 'G', 'H'],
    'F': ['C', 'H'],
    'G': ['D', 'E', 'S'],
    'H': ['E', 'F', 'S'],
    'S': ['G', 'H']
}

# Run BFS starting from node 'A'
visit_sequence = bfs(graph, 'A')
print("Visit sequence:", visit_sequence)
```



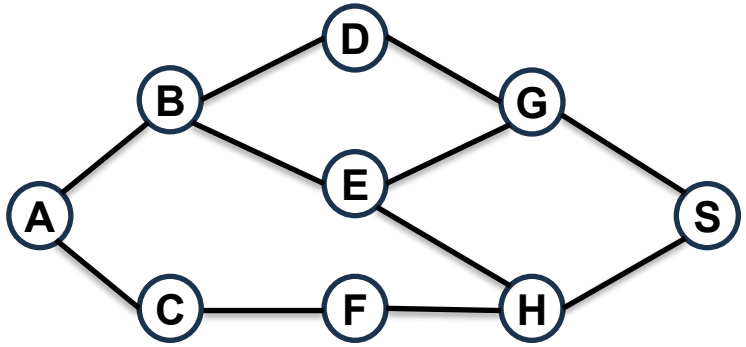

```
Visit_sequence: ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'S']
```

(The order of nodes visited may differ depending on the order neighbors are processed.)

Q3. You are given the following undirected graph.

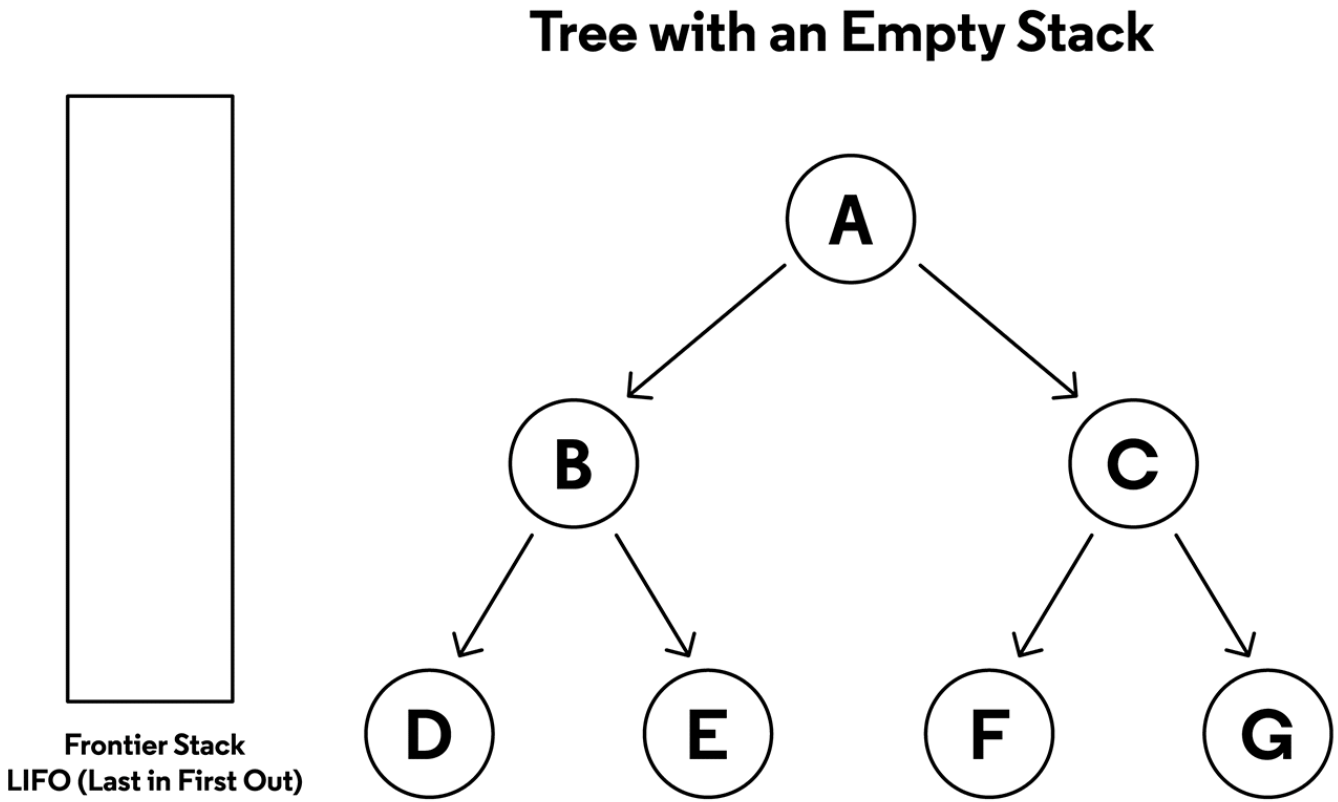
(i) Write a Python program to perform a Breadth-First Search (BFS) traversal starting from node ‘A’. Print the order in which nodes are visited.

(ii) Write a Python program to perform a Depth-First Search (DFS) traversal starting from node ‘A’. Print the order in which nodes are visited.



Note: The order of nodes visited may differ depending on the order neighbors are processed.

Recap: DFS



Credit: https://www.codecademy.com/article/depth-first-search-conceptual

Q3. (ii) DFS

```python
# Define DFS function using stack
def dfs(graph, start):
    visited = []   # List to keep track of visited nodes
    stack = []     # Stack to hold nodes to explore
    sequence = []  # List to store the visiting sequence

    visited.append(start)
    stack.append(start)

    while stack:
        node = stack.pop()
        print(node)
        sequence.append(node)

        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.append(neighbour)
                stack.append(neighbour)

    return sequence
```

Create an empty list to keep track of explored (visited) nodes
Create an empty list serve as the frontier, which acts as a **stack**
Create an empty list to store the visited nodes *in the order they are explored*

Add the starting node to the frontier (stack) and mark it as visited

While frontier (stack) is not empty, repeat:

-- Pop the most recently added node from the stack (**LIFO order**).

-- add it to the sequence and start to expand the node

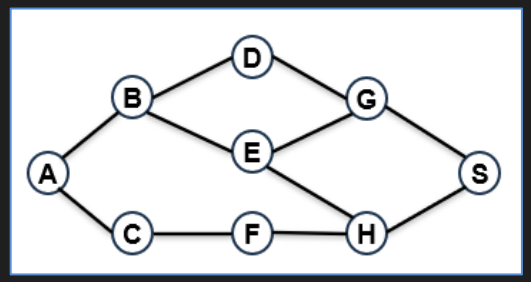
-- **For each neighbor of the node**:

- If the neighbor has not been visited,
- Mark it as visited and push it onto the stack.

Q3. (ii) DFS (cont.)

```python
# Define the graph using a dictionary.
# Each node maps to a list of its connected neighbors.
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B', 'G'],
    'E': ['B', 'G', 'H'],
    'F': ['C', 'H'],
    'G': ['D', 'E', 'S'],
    'H': ['E', 'F', 'S'],
    'S': ['G', 'H']
}

# Call DFS starting from node 'A'
visit_sequence = dfs(graph, 'A')
print("Visit_sequence:", visit_sequence)
```




```
Visit_sequence: ['A', 'C', 'F', 'H', 'S', 'G', 'D', 'E', 'B']
```

(DFS doesn't guarantee a unique order - it depends on the order neighbors that are processed)

Q3. (ii) DFS (recursive method)

```python
# Recursive DFS function
def dfs(graph, node, visited):
    print(node)
    visited.append(node)

    for neighbour in graph[node]:
        if neighbour not in visited:
            dfs(graph, neighbour, visited)
```

Recursion: call the same dfs function **on the unvisited neighbor**. This means we go deeper before we come back up, **a core idea of depth-first search.**

- Visits nodes as deep as possible before backtracking.
- The visited list is passed along to remember which nodes we've already explored.
- Here, **recursion** handles the **stack-like behavior** of DFS automatically.

Q3 Code highlights: List operations in Python

```
visited = []
queue = []
sequence = []
```
Create empty lists

append()- Add an item to a list

- **Adds an element to the end of a list.**
- **Commonly used to add nodes to the visited list or search frontier (queue/stack).**

```
visited.append(start)
```
Adding the start node to the visited list

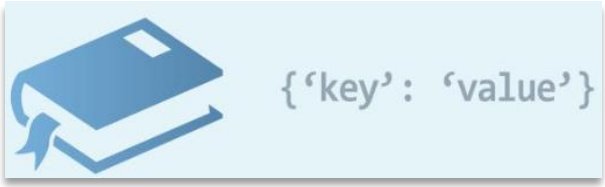
pop()- Remove and return an item from a list

- **pop(0) removes the first item → used in queues (FIFO)**
- **pop() (without index) removes the last item → used in stacks (LIFO)**

```
node = queue.pop(0)
```
Removes the earliest added node from the queue

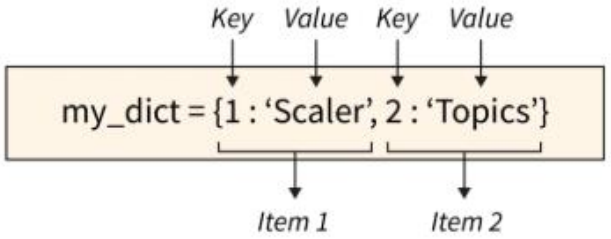
Reference: https://docs.python.org/3/tutorial/datastructures.html

Q3 Code highlights: Python dictionary



A Python dictionary is a collection of key-value pairs used to store and quickly access data by unique keys.

```
my_dict = {}           # create an empty dictionary
my_dict[1] = 'Scaler'  # assign the value 'scalar' to key 1
my_dict[2] = 'Topics'  # get the value 'Topics' to key 2
```




```
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B', 'G'],
    'E': ['B', 'G', 'H'],
    'F': ['C', 'H'],
    'G': ['D', 'E', 'S'],
    'H': ['E', 'F', 'S'],
    'S': ['G', 'H']
}
```

- In search algorithms, we often use a dictionary to represent a graph.
- Each key represents a node (e.g., 'A', 'B'), and
- each value is a list of that node's direct neighbors (i.e., the nodes it connects to directly).
- We also commonly use dictionary to record how we reach each node during traversal. E.g., each key is a node, and each value is the predecessor node from which that node was reached.

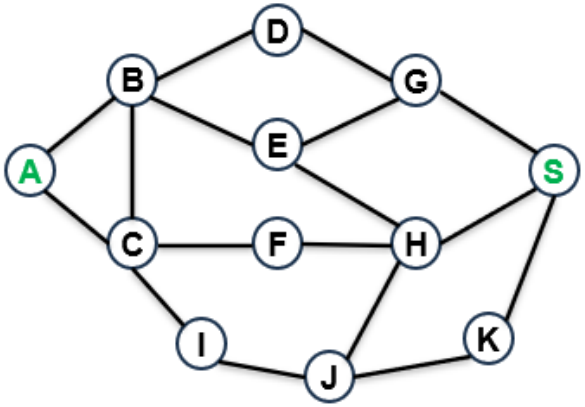
Reference: https://docs.python.org/3/tutorial/datastructures.html#dictionaries

Q4. Your friend is flying from Arequipa (A), a historic city and the second-largest in Peru, to Singapore (S). However, there is no direct flight between the two cities.

You decide to plan a route with the minimum number of flight segments needed for your friend. To do this, you model the available flight connections as an undirected graph as shown below, where nodes represent cities, and edges represent direct flights between cities. The modeled graph is shown below.

(i) Which strategy is better for finding the fewest number of flight segments: Breadth-First Search (BFS) or Depth-First Search (DFS)?

(ii) Write a Python programme to compute the minimum number of flight segments from Arequipa to Singapore.

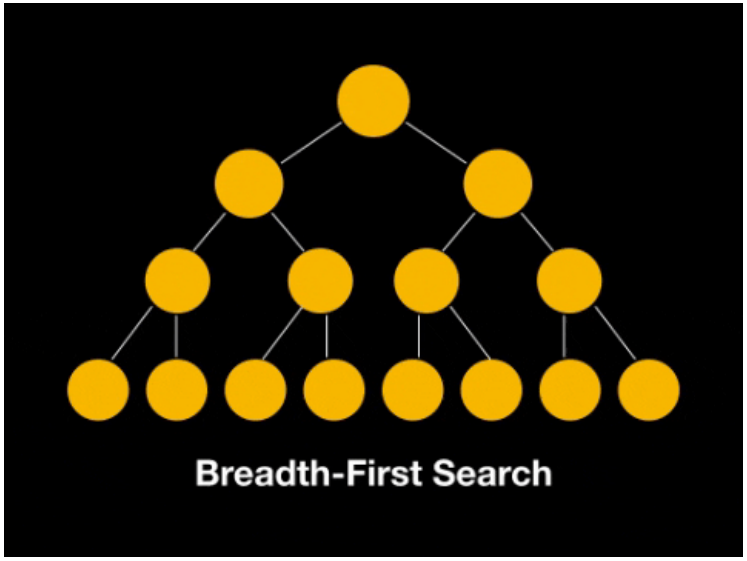


Reference: A similar but tougher question is LeetCode #127: https://leetcode.com/problems/word-ladder/description/

Q4. (i) Which strategy is better for finding the fewest number of flight segments: Breadth-First Search (BFS) or Depth-First Search (DFS)?

Why BFS?

- BFS explores all cities that are 1 flight away, then 2 flights away, and so on.
- It guarantees that the first time we reach the destination, it's through the **shortest path (least number of flights)**.
- Think of it as searching **level by level.**



Why not DFS?

DFS explores deeply first, possibly following long or inefficient paths.
It might reach the destination later, not guaranteeing the shortest route.

Source: https://medium.com/@basabjha/dsa-graph-secrets-dfs-and-bfs-made-simple-for-beginners-4532f90d7611

(ii) Write a Python function to compute the minimum number of flight segments from Arequipa to Singapore.

```python
def bfs_shortest_path(graph, start, goal):
    visited = []  # List to keep track of cities we have already visited
    queue = []    # Queue for BFS: stores cities to explore next
    path = {}

    queue.append(start)
    visited.append(start)
    path[start] = None

    while queue:
        city = queue.pop(0)

        if city == goal:
            result = []  # List to store the path from start to goal
            while city is not None:
                result.append(city)
                city = path[city]
            result.reverse()  # reverse to get path from start to goal
            return result

        for neighbor in graph[city]:
            if neighbor not in visited:
                visited.append(neighbor)
                queue.append(neighbor)
                path[neighbor] = city

    return None  # if no path found
```

Create an empty dictionary to remember how we got to each city
Each **key** is a **city**, and its **value** is the **city we came from** (its **predecessor**).
path = {city: predecessor}

Start node has no predecessor

Loop until there are no more cities to explore

Remove (pop) the **earliest** city added to the queue to explore (**FIFO order**)

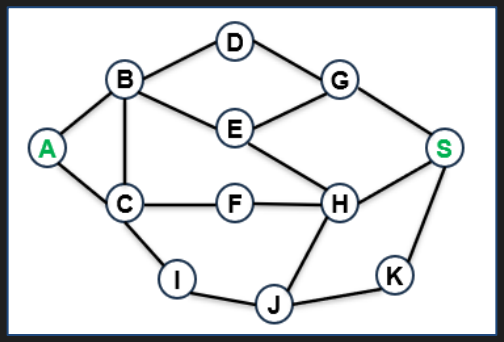
- If the current city == goal, **reconstruct the path** taken to reach it.
- We do this by following each city's **predecessor** in the path dictionary.

Check all neighboring cities directly connected to the current city.
If a neighbor has not been visited:

- Mark it as visited
- Add it to the queue
- Record its **predecessor** (i.e. the **current city**) in the path dictionary

(ii) Write a Python function to compute the minimum number of flight segments from Arequipa to Singapore (cont.).

```
# Define the graph using a dictionary.
# Each city (node) maps to a list of cities it has direct flights to (neighbors).
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'C', 'D', 'E'],
    'C': ['A', 'B', 'F', 'I'],
    'D': ['B', 'G'],
    'E': ['B', 'G', 'H'],
    'F': ['C', 'H'],
    'G': ['D', 'E', 'S'],
    'H': ['E', 'F', 'J', 'S'],
    'I': ['C', 'J'],
    'J': ['I', 'K'],
    'K': ['J', 'S'],
    'S': ['G', 'H']
}
```

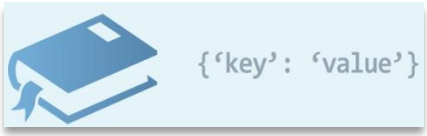


Note: There can be more than one path with the same minimum number of flight segments, and our program returns just one of them

```
# Find the shortest path using bfs
route = bfs_shortest_path(graph, 'A', 'S')
if route:
    print("Path with the fewest flight transfers from Arequipa to Singapore:", route)
    print("Minimum number of flight segments:", len(route) - 1)
else:
    print("No route found.")
```

```
Path with the fewest flight transfers from Arequipa to Singapore: ['A', 'B', 'D', 'G', 'S']
Minimum number of flight segments: 4
```

Q4 Code highlights: Dictionary operations



`path = {city: predecessor}`

Dictionary structure to track how we reach each city
It allows us to **reconstruct the entire route** from the start city to the goal by backtracking from the goal using this stored information.

```
path[neighbor] = city
```

Dictionary Assignment
Records the **current city** as the **predecessor** of the **neighbor.**
Helping us remember how we reached each node during the search.

```
city = path[city]
```

Dictionary Lookup
Retrieves the **predecessor** of the current city from the path dictionary, and assigns it back to city.
Used to trace the path backward from goal to start.

Reference: https://docs.python.org/3/tutorial/datastructures.html#dictionaries

Q4 Code highlights: List operations

```
result = reverse()
```

Reversing a list

`.reverse()` reverse the order of elements in the list.
Modifies the original list (does not create a new list)

In pathfinding algorithms like BFS, we often build the path backward (from goal to start).

We use .reverse() to flip it and get the correct order: from start to goal.

Reference: https://docs.python.org/3/tutorial/datastructures.html

Q4 Takeaway

Breadth-First Search (BFS) is the most suitable strategy for finding the fewest number of flight segments, as it explores all paths layer by layer and guarantees the shortest path in terms of edge count in unweighted graphs.

Multiple equally optimal solutions may exist: There can be more than one path with the same minimum number of flight segments. Since in our program, BFS stops when it finds the first shortest path, it returns just one of these equally valid solutions.

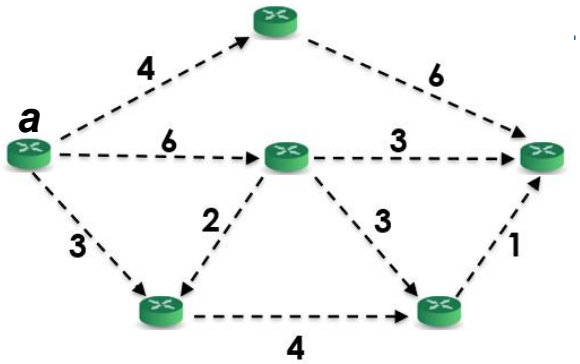
Reference: https://docs.python.org/3/tutorial/datastructures.html

Q5. Signal Propagation in a Network (Adapted from LeetCode #743)

You are given a communication network consisting of several nodes connected by directed links, each with an associated transmission delay (indicated by the numbers beside each edge), as illustrated in the graph below. Given a starting node a (the leftmost node), determine the minimum time required for a signal to reach all nodes in the network.

i. Which search algorithm that you have learned so far would be most suitable for this problem? Briefly justify your choice.

ii. (b) Implement your solution in Python using the selected search algorithm. Your program should:
   - Model the network as a graph.
   - Compute the minimum time it takes for the



Q5 (i) Which search algorithm that you have learned so far would be most suitable for this problem?

Algorithm	Handles weights	Finds optimal Path	Uses heuristic	Complete	Best use case
BFS	No	Yes (unweighted)	No	Yes	Shortest path in **unweighted** graphs
DFS	No	No	No	No	Explores deeply, not optimal paths
Dijkstra	Yes	Yes	No	Yes	**Best for shortest paths with weights**
Greedy Best-First	Yes	No	Yes	No	Fast guesses, not always correct
A*	Yes	Yes (if heuristic is good)	Yes	Yes	Combines cost + heuristic

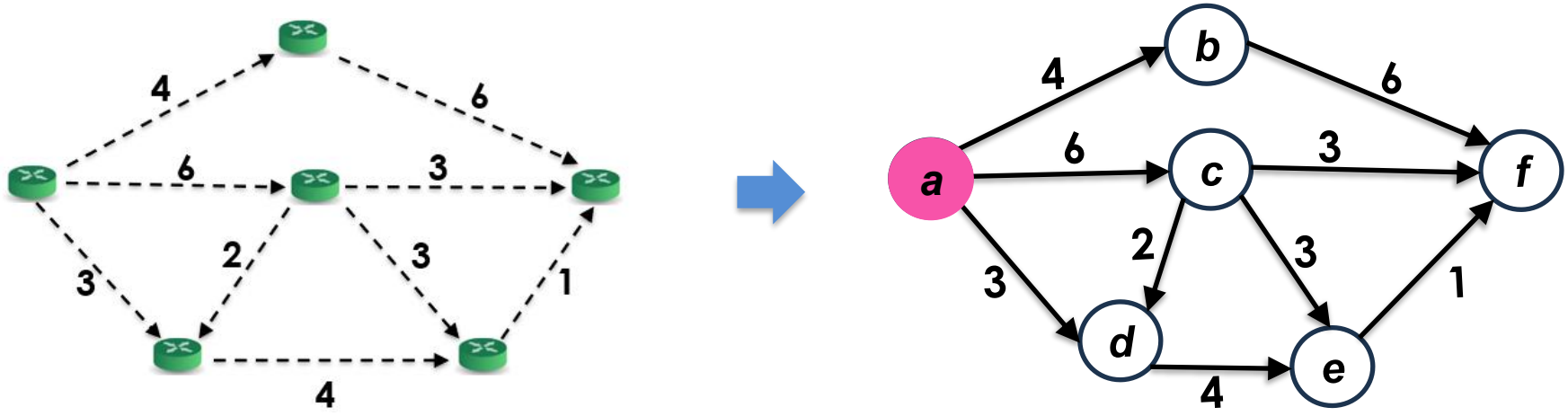
Q5 (i) Which search algorithm that you have learned so far would be most suitable for this problem? (cont.)

The most suitable algorithm is **Dijkstra's Algorithm**.

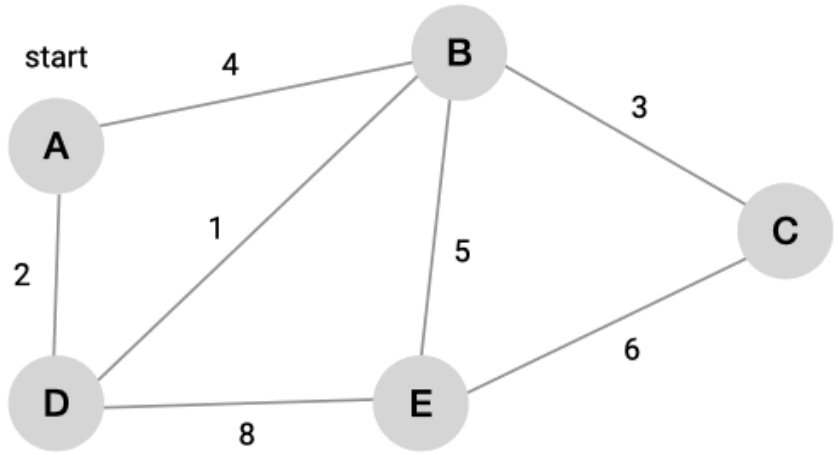
Justification: Dijkstra's algorithm is specifically designed to find the shortest paths from a single source node to all other nodes in a graph with non-negative edge weights. It works by greedily selecting the unvisited node with the smallest known distance from the source, ensuring that the first time it reaches a node, it has found the shortest path to it.

Q5 (ii) Implement your solution in Python using the selected search algorithm.

Model the communication network as a graph and specify the starting node.



Recap: Dijkstra's algorithm



Vertex	Shortest distance from A	Previous vertex
A	0	
B	∞	
C	∞	
D	∞	
E	∞	

Credit: https://memgraph.com/docs/advanced-algorithms/deep-path-traversal

Dijkstra's algorithm: Pseudocode

Dijkstra(G, l, s)

for all $u \in V$:

$dist[u] \leftarrow \infty$

$prev[u] \leftarrow nil$

$X \leftarrow \emptyset$ // X: processed vertices

$Q \leftarrow \emptyset$ // Q: frontier, implemented as priority queue

$dist[s] \leftarrow 0$

updateQueue $(Q, s, 0)$ // enqueue starting node with distance 0

while Q is not empty:

$u \leftarrow$ extractMin (Q) // remove and return node with min distance

$X \leftarrow X \cup \{u\}$ // mark u as processed

for all edges $(u, v) \in E$:

if $v \notin X$ and $dist[u] + l(u, v) < dist[v]$:

$dist[v] \leftarrow dist[u] + l(u, v)$ // update shortest distance

$prev[v] \leftarrow u$ // update previous node

updateQueue $(Q, v, dist[v])$ // update priority queue

Case1: a node isn't in the queue, it's inserted with its distance.
Case2: a node is in the queue and a shorter path is found, update the distance

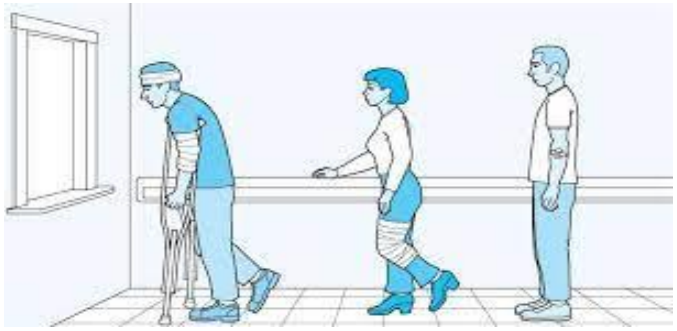
Q5 (ii) Implement your solution in Python using the selected search algorithm.

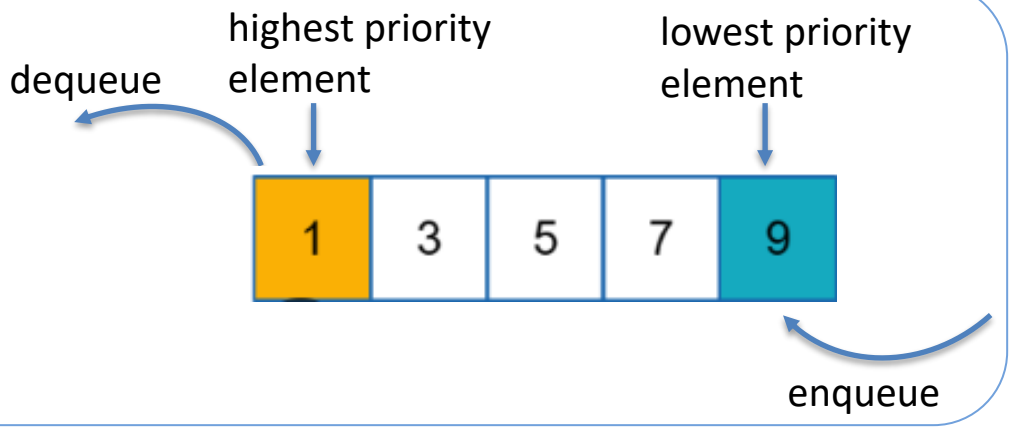
Key implementation detail:

•Dijkstra's algorithm uses a **priority queue** to select the nearest unvisited node

•In Python, a **heap-based priority queue** (like heapq) is the most common and efficient way to implement Dijkstra's algorithm, though other structures are also possible.

Priority queue:





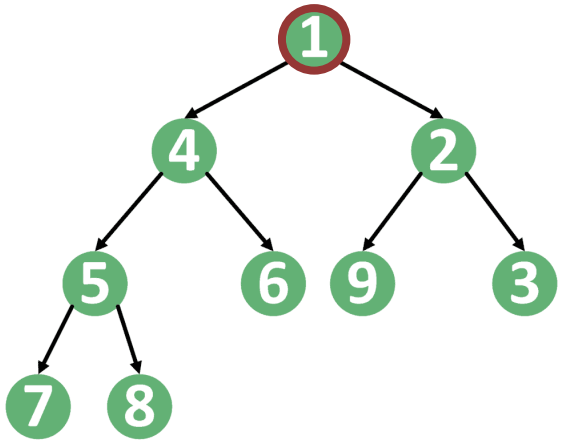
Overview: Heap data structure

- A heap is a data structure designed to quickly find and update the minimum/maximum element

- Binary min-heap:

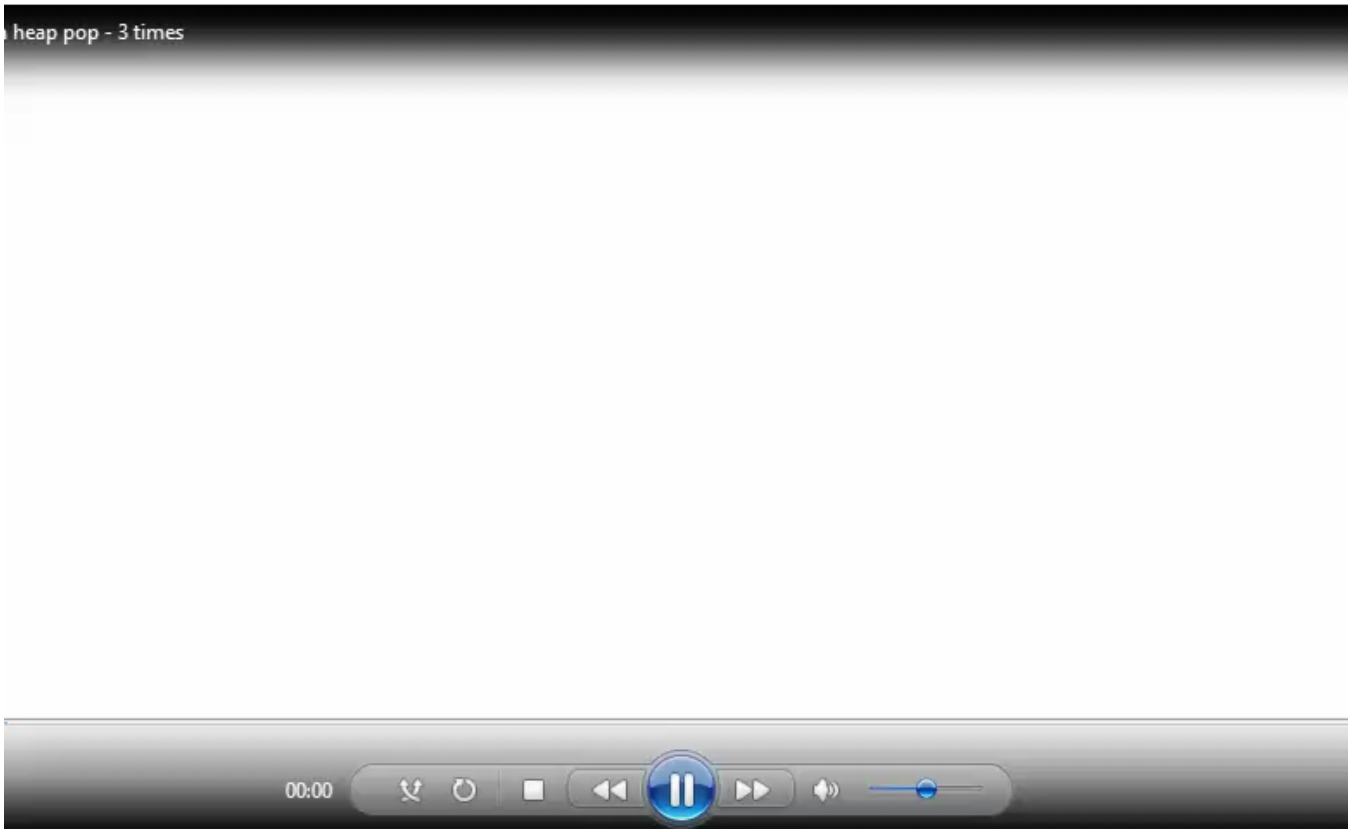
  -- a complete binary tree.

  -- at every node, key ≤ children's keys



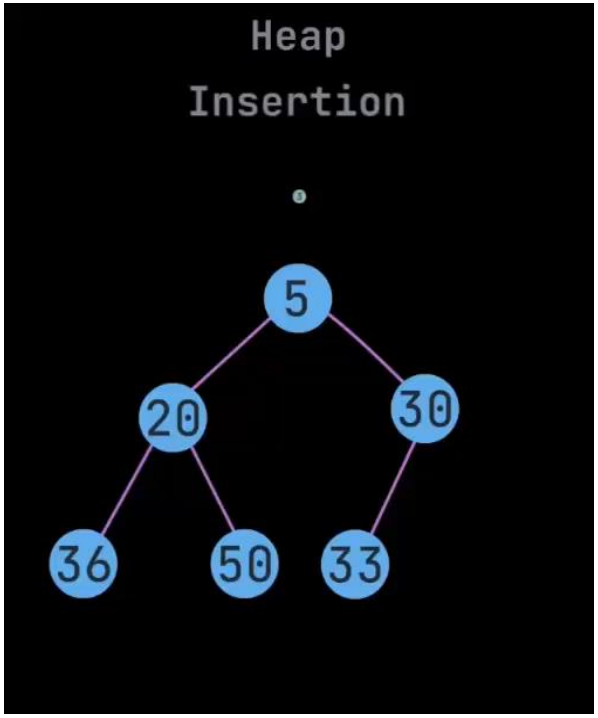
Overview: Heap operation

Extract the minimum from a min-heap by swapping up last leaf, bubbling down:

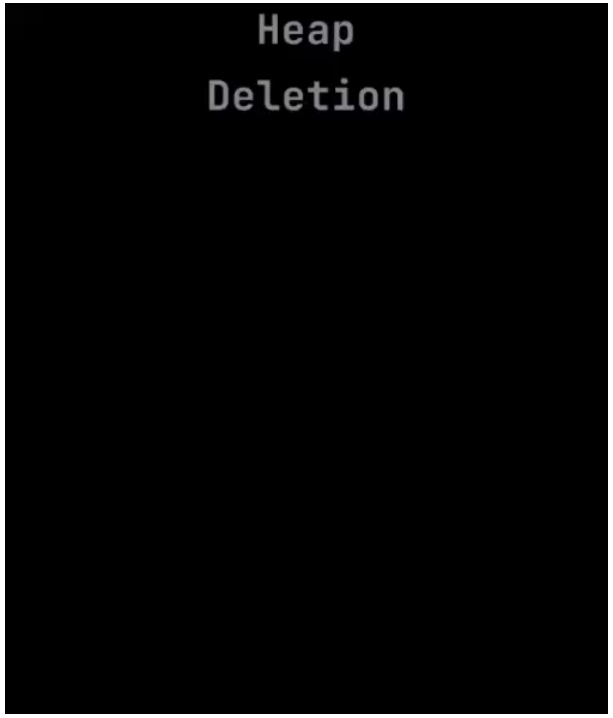


Reference: https://www.youtube.com/shorts/3Q4oQk4pEks

Overview: Heap operation (cont.)



Insertion in min-heap



Deletion in min-heap

- In our class, you don't need to understand how a heap works internally.
- Python provides a built-in library (heapq) that makes it easy to use.
- Just remember: a **heap** is a data structure used for a **priority queue**, helping Dijkstra's algorithm efficiently *select the node with the smallest distance*.

Source: https://www.youtube.com/shorts/QAaxJmdneS4

Q5 (ii) Implement your solution in Python

```python
from heapq import heappush, heappop
import math

def dijkstra(graph, s):
    node_data = {node: {'dist': math.inf, 'prev': []} for node in graph}
    X = set()  # processed/visited nodes
    Q = []     # frontier, implemented as a priority queue

    heappush(Q, (0, s))  # enqueue starting node (source) s with distance 0
    node_data[s]['dist'] = 0

    while Q:
        u_dist, u = heappop(Q)  # extractMin(Q)
        if u in X:
            continue
        X.add(u)  # mark u as processed

        for v in graph[u]:
            if v not in X:
                dist = node_data[u]['dist'] + graph[u][v]
                if dist < node_data[v]['dist']:
                    node_data[v]['dist'] = dist  # update shortest distance
                    node_data[v]['prev'] = node_data[u]['prev'] + [u]
                    heappush(Q, (node_data[v]['dist'], v))  # updateQueue

    return node_data
```

for all $u \in V$:
  $dist[u] \leftarrow \infty$
  $prev[u] \leftarrow nil$
$X \leftarrow \emptyset$
$Q \leftarrow \emptyset$
$dist[s] \leftarrow 0$
updateQueue $(Q, s, 0)$

while Q is not empty:
  $u \leftarrow$ extractMin (Q)

$X \leftarrow X \cup \{u\}$

for all edges $(u, v) \in E$:
  if $v \notin X$ and $dist[u] + l(u, v) < dist[v]$:
    $dist[v] \leftarrow dist[u] + l(u, v)$
    $prev[v] \leftarrow u$
    updateQueue $(Q, v, dist[v])$

Q5 (ii) Implement your

```
node_data = {
   'a': {'dist': 0, 'prev': []} ,
   'b': {'dist': 4, 'prev': ['a']} ,
   'c': {'dist': 6, 'prev': ['a']} ,
   'd': {'dist': 3, 'prev': ['a']} ,
   'e': {'dist': 7, 'prev': ['a', 'd']} ,
   'f': {'dist': 8, 'prev': ['a', 'd', 'e']} ,
}
```

node	shortest distance from *a*	previous node
a	0	--
b	4	*a*
c	6	*a*
d	3	*a*
e	7	*d*
f	8	*e*

```
from heapq import heappush, heappop
import math

def dijkstra(graph, s):
    node_data = {node: {'dist': math.inf, 'prev': []} for node in graph}
    X = set()  # processed/visited nodes
    Q = []     # frontier

    heappush(Q, (0, s))  # enqueue starting node (source) s with distance 0
    node_data[s]['dist'] = 0

    while Q:
        u_dist, u = heappop(Q)  # extractMin(Q)
        if u in X:
            continue
        X.add(u)  # mark u as processed

        for v in graph[u]:
            if v not in X:
                dist = node_data[u]['dist'] + graph[u][v]
                if dist < node_data[v]['dist']:
                    node_data[v]['dist'] = dist  # update shortest distance
                    node_data[v]['prev'] = node_data[u]['prev'] + [u]
                    heappush(Q, (node_data[v]['dist'], v))  # updateQueue

    return node_data
```

Create a **dictionary** to store:
- dist – current shortest distance from the source (initial value set to $+\infty$)
- prev – previous nodes on the shortest path

Q helps us always pick the unprocessed node that is currently closest to the source.

- Create a priority queue implemented using a min heap
- Push the source node s into the priority queue with distance 0

Distance from source to itself is 0.

- Get the node with the smallest distance from the source
- Q is a priority queue (min heap), so heappop() always returns the node with the smallest distance

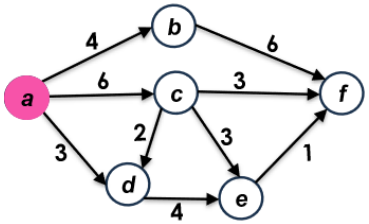
For all neighbouring nodes that are not processed:

If going through u gives a **shorter distance** to v:

- Update the shortest distance and remember the path taken
- Push node v into the priority queue with its new distance

node_data contains the shortest path and distance from s to every node

Q5 (ii) Implement your solution in Python (cont.)



Inside result dictionary:

```
result = {
  'a': {'dist': 0, 'prev': []} ,
  'b': {'dist': 4, 'prev': ['a']} ,
  'c': {'dist': 6, 'prev': ['a']} ,
  'd': {'dist': 3, 'prev': ['a']} ,
  'e': {'dist': 7, 'prev': ['a', 'd']} ,
  'f': {'dist': 8, 'prev': ['a', 'd', 'e']} ,
}
```

Output:

```
Shortest Distance to a : 0
Shortest Path to a : ['a']
Shortest Distance to b : 4
Shortest Path to b : ['a', 'b']
Shortest Distance to c : 6
Shortest Path to c : ['a', 'c']
Shortest Distance to d : 3
Shortest Path to d : ['a', 'd']
Shortest Distance to e : 7
Shortest Path to e : ['a', 'd', 'e']
Shortest Distance to f : 8
Shortest Path to f : ['a', 'd', 'e', 'f']
```

```
# Graph represented as an adjacency list (dictionary of dictionaries)
graph = {
    'a': {'b': 4, 'c': 6, 'd': 3},
    'b': {'f': 6},
    'c': {'d': 2, 'e': 3, 'f': 3},
    'd': {'e': 4},
    'e': {'f': 1},
    'f': {},
}

source = 'a'
result = dijkstra(graph, source)

# Print shortest distances from the source to all other nodes
for node in result:
    dist = result[node]['dist']
    path = result[node]['prev'] + [node]   # build the final path
    print("Shortest Distance to", node, ":", dist)
    print("Shortest Path to", node, ":", path)
```

- The graph is stored as a **dictionary of dictionaries**
- Each key (like 'a') is a node, and its value is another dictionary listing neighbors and their edge weights
- For example, 'a': {'b': 4, 'c': 6, 'd': 3} means node 'a' connects to 'b' with cost 4, 'c' with cost 6, and 'd' with cost 3

There may be more than one shortest path with the same total distance — Dijkstra's algorithm returns just one of them.

For every node in result:

- Get the shortest distance from the source
- Get the full path from the source to that node
- Print both the distance and the path clearly

Q5 Code highlights – Python libraries

```
from heapq import heappush, heappop
import math
```

•**heapq**: A library for **priority queue** operations using a **heap** data structure

•**math**: A fundamental Python package for **mathematical functions**, supporting basic and advanced math operations

```
node_data = {node: {'dist': math.inf, 'prev': []} for node in
graph}
```

Set the initial shortest distances from the source to every node as positive infinity (math.inf).

vertex	shortest distance from *a*	previous vertex
a	∞	--
b	∞	--
c	∞	--
d	∞	--
e	∞	--
f	∞	--

Q5 Code highlights – heapq library

```
from heapq import heappush, heappop
```

heappush(heap, item) is a function that adds an element (item) to the heap (heap) while maintaining the heap property.

The heap is a list that stores elements arranged so that the smallest element is always at the front (index 0).

Parameters:
heap: A list representing the heap (priority queue).
item: The element you want to add to the heap.

For example:
heappush(Q, (0, s)) push the tuple (0, s) into the heap named Q.

For more information, check the heapq documentation: https://docs.python.org/3/library/heapq.html

Q5 Code highlights – heapq library (cont.)

```
from heapq import heappush, heappop
```

heappop(heap) is a function that **removes and returns the smallest element** from the heap while maintaining the heap property.

The heap is a list that stores elements arranged so that the smallest element is always at the front (index 0).

Parameter:
heap: A list representing the heap (priority queue).

For example:
heappop(Q) removes and returns the smallest element from the heap named Q.

For more information, check the heapq documentation: https://docs.python.org/3/library/heapq.html

Q5 Code highlights – heapq library (cont.)

```
from heapq import heappush, heappop
```

heappop(heap) is a function that **removes and returns the smallest element** from the heap while maintaining the heap property.

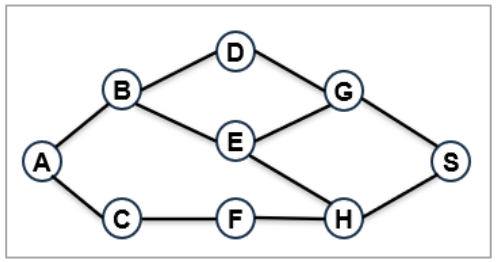
The heap is a list that stores elements arranged so that the smallest element is always at the front (index 0).

Parameter:
heap: A list representing the heap (priority queue).

For example:
heappush(Q) removes and returns the smallest element from the heap named Q.

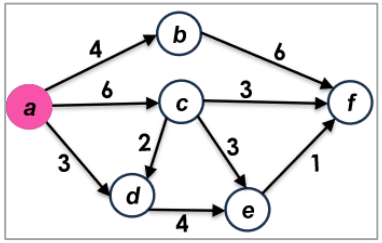
For more information, check the heapq documentation: https://docs.python.org/3/library/heapq.html

Q5 Code highlights – Graph representations in Python




```
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B', 'G'],
    'E': ['B', 'G', 'H'],
    'F': ['C', 'H'],
    'G': ['D', 'E', 'S'],
    'H': ['E', 'F', 'S'],
    'S': ['G', 'H']
}
```

Unweighted (directed) graph:
Represented as a **dictionary** where each *key* is a node, and the *value* is a **list** of directly connected neighbors.



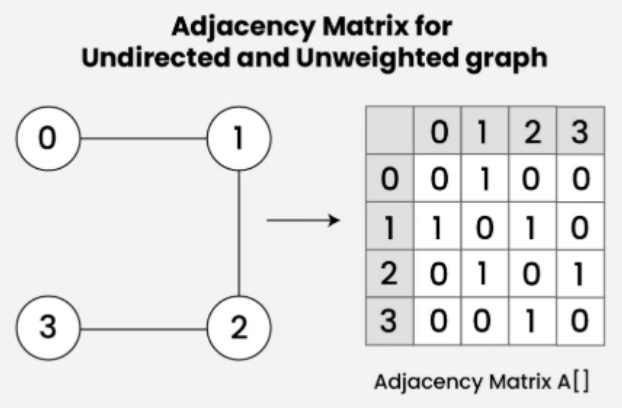

```
graph = {
    'a': {'b': 4, 'c': 6, 'd': 3},
    'b': {'f': 6},
    'c': {'d': 2, 'e': 3, 'f': 3},
    'd': {'e': 4},
    'e': {'f': 1},
    'f': {},
}
```

Weighted (directed) graph:
Represented as a **dictionary of dictionaries** (adjacency list)**:**

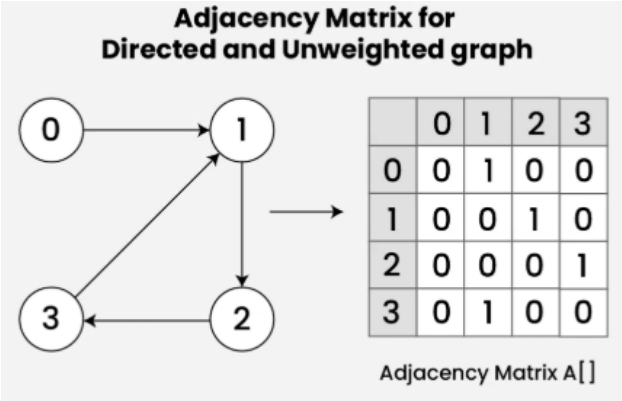
- Each *key* is a node.
- The *value* is another dictionary, where:
  - Each *key* is a **neighboring node**, and
  - Each *value* is the **weight** of the edge to that neighbor.

Q5 Code highlights – Graph representations in Python (cont.)

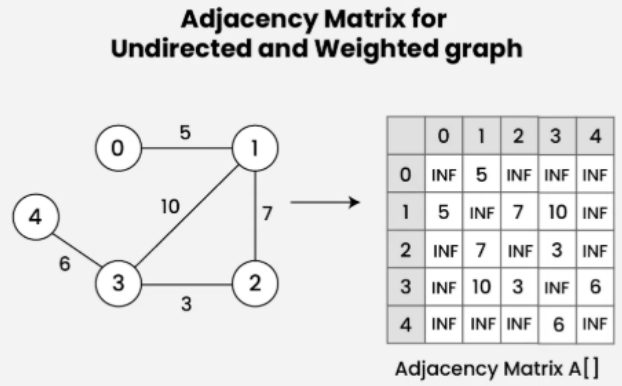
Representing graph with adjacency matrix



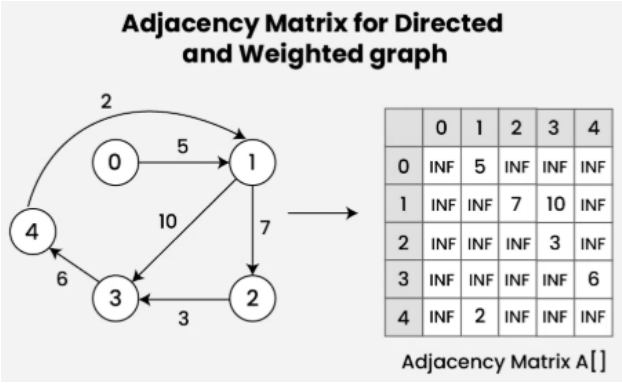
	0	1	2	3
0	0	1	0	0
1	1	0	1	0
2	0	1	0	1
3	0	0	1	0



	0	1	2	3
0	0	1	0	0
1	0	0	1	0
2	0	0	0	1
3	0	1	0	0



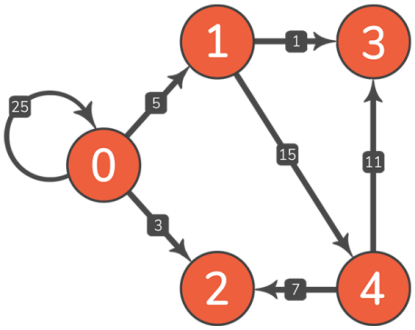
	0	1	2	3	4
0	INF	5	INF	INF	INF
1	5	INF	7	10	INF
2	INF	7	INF	3	INF
3	INF	10	3	INF	6
4	INF	INF	INF	6	INF



	0	1	2	3	4
0	INF	5	INF	INF	INF
1	INF	INF	7	10	INF
2	INF	INF	INF	3	INF
3	INF	INF	INF	INF	6
4	INF	2	INF	INF	INF

Reference: https://www.geeksforgeeks.org/dsa/adjacency-matrix/

Q5 Code highlights – Graph representations in Python (cont.)

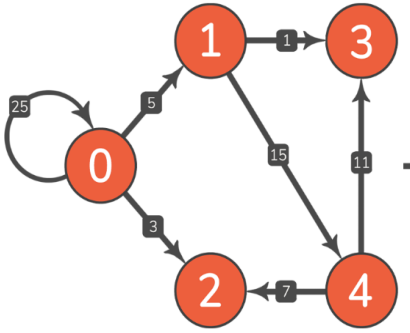


Dictionary with values assigned

weight

```
{  0: {(0, 25), (1, 5), (2, 3)},
   1: {(3, 2), (4, 15)},
   2: {},
   3: {},
   4: {(2, 7), (3, 11)},
}
```

Adjacency list/dictionary



	0	1	2	3	4
0	25	5	3	0	0
1	0	0	0	1	15
2	0	0	0	0	0
3	0	0	0	0	0
4	0	0	7	11	0

Adjacency matrix

Representation	Adjacency list/Dictionary	Adjacency matrix
Best for	Sparse graphs	Dense graphs
Traversal	Efficient (only actual neighbors)	Slower (scan all possible neighbors)
Space efficiency	Efficient: Uses less memory for sparse graphs	Consumes more space
Graph updates	Easy to add/remove nodes and edges	Less flexible (matrix resizing needed)
Typical use cases	Preferred for most real-world problems, like road maps or networks.	Dense networks (e.g., social network of close-knit friends), fixed-size graphs

Reference: https://stackabuse.com/courses/graphs-in-python-theory-and-implementation/lessons/representing-graphs-in-code/

Q5 Code highlights: Dictionary operations

```
node_data = {node: {'dist': math.inf, 'prev': []} for node in graph}
```

Initializes a dictionary called node_data to store information about each node in the graph. It's used to track:

- The shortest distance from the source node to each node ('dist')
- The previous nodes in the shortest path ('prev')

Code break down:

`for node in graph`: **Loops** through all nodes in the graph. For each node, create an entry in the `node_data` dictionary:

-- **key:** the node itself.

-- **value:** another **dictionary** `{'dist': math.inf, 'prev': []}`:

- `'dist': math.inf` –initializes the distance to infinity, meaning the node is initially unreachable.
- `'prev': []` – starts with an empty path. This list will later be used to reconstruct the shortest path.

node	shortest distance from *a*	previous node
a	0	--
b	4	*a*
c	6	*a*
d	3	*a*
e	7	*d*
f	8	*e*

`node_data` after the search completes contains the shortest distance and path from the source to every node:

```
node_data = {
  'a': {'dist': 0, 'prev': []} ,
  'b': {'dist': 4, 'prev': ['a']} ,
  'c': {'dist': 6, 'prev': ['a']} ,
  'd': {'dist': 3, 'prev': ['a']} ,
  'e': {'dist': 7, 'prev': ['a', 'd']} ,
  'f': {'dist': 8, 'prev': ['a', 'd', 'e']} ,
}
```

Q5 Code highlights: Dictionary operations

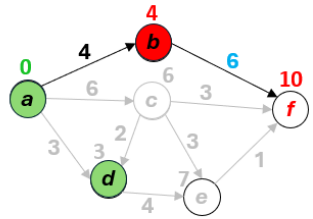
`dist = node_data[u]['dist'] + graph[u][v]`

Calculates the total distance from the source to node v, by going through node u.

`node_data[u]['dist']`: known shortest distance from the source to u

`graph[u][v]`: distance of the edge from u to v

Their sum gives the distance from the source to v via u



$$dist[f] = dist[b] + 6$$
$$= 4 + 6$$
$$= 10$$

Nested dictionary:

`node_data[u]['dist']`: u refers to the current node, and 'dist' accesses the shortest distance from the source to that node, stored in the *inner dictionary*.

`graph[u][v]`: u is the current node, and v is one of its neighbors in the *inner dictionary* (the value of u).

node_data after the search completes contains the shortest distance and path from the source to every node:

```
node_data = {
  'a': {'dist': 0, 'prev': []} ,
  'b': {'dist': 4, 'prev': ['a']} ,
  'c': {'dist': 6, 'prev': ['a']} ,
  'd': {'dist': 3, 'prev': ['a']} ,
  'e': {'dist': 7, 'prev': ['a', 'd']} ,
  'f': {'dist': 8, 'prev': ['a', 'd', 'e']} ,
}
```

```
graph = {
    'a': {'b': 4, 'c': 6, 'd': 3},
    'b': {'f': 6},
    'c': {'d': 2, 'e': 3, 'f': 3},
    'd': {'e': 4},
    'e': {'f': 1},
    'f': {},
}
```

Q5 Takeaway

Graph Representation:
The choice between an *adjacency list* and an *adjacency matrix* depends on the graph's density.

- Use an adjacency list for sparse graphs (with fewer edges) — it's more memory-efficient.
- Use an adjacency matrix for dense graphs (with many edges) — it allows faster edge lookups.

Implementation Shortcuts:
In practice, we sometimes deviate slightly from the textbook version of an algorithm to prioritize efficiency and speed.
For example, in Dijkstra's algorithm, we may allow the same node to enter the frontier multiple times, instead of updating its position in the queue — this avoids the overhead of updating.
These trade-offs prioritize efficiency while still *guaranteeing* the algorithm's *correctness*.

Want to explore more?

A similar but more challenging problem:

LeetCode #787: Cheapest Flights Within K Stops
https://leetcode.com/problems/cheapest-flights-within-k-stops/description/

- Model cities as nodes and flights (with cost) as directed weighted edges
- Find the cheapest path from source to destination within at most K stops
- Similar to minimizing signal delay across a network with constraints
- Can use a modified Dijkstra's algorithm or other graph traversal methods considering cost and stop limits.

Try it in your spare time if you're comfortable with today's material and want an extra challenge!

Reference of LeetCode #787:
https://www.youtube.com/watch?v=vWgoPTvQ3Rw
https://medium.com/@hongjje.dev/leetcode-solution-787-cheapest-flights-within-k-stops-321ca7929fa9